1. **(20 pt) Gradient Descent:** You are given the paired measured data points on the right, and you are asked to find the local minima numerically using gradient descent.
    a. Plot the data
    b. Assume learning rate is 0.1
    c. Start at x(1) = 0.9, using gradient descent and find the value of x(k), where f(x(k)) is minimum.
    d. What to show:
        i. Create a table of k, x(k), estimated derivative at k (1, 7), and f(x(k))
        ii. Start at x = 0.9, apply gradient descent and find the local minima. Show each gradient descent with a red dot on the plot.
        iii. Now try a learning rate of 0.5 and see what happens to the solution

x	f(x) (measured)
0.8	-1.784
0.9	-1.952
1.0	-2.080
1.1	-2.168
1.2	-2.216
1.3	-2.224
1.4	-2.192
1.5	-2.120
1.6	-2.008

2. **(30) Anisotropic Diffusion:** You are asked to denoise an image using "Total Variation" (see your slides).
    a. Use "Synthetic_image" in your file system
    b. Add noise with PSNR of 18.4dB.
        i. Show the code and the process that you followed
        ii. Show the noisy image next to the original
    c. Smooth the noisy image with a Gaussian filter with sigma = 2
    d. Apply total variations. Your options are
        i. tv_diffusion.py
        ii. denoise_tv_bregman, which has an isotropic=True/False
        iii. There is also perona_malik() for comparison.
        iv. Compare Total Variation and Gaussian and report.
        v. Apply Total Variation to the nature_creek image. If the code does not accept RGB images, feed each channel independently and reconstruct back.
3. **(30) Experiment with connected components:** Examine the image named "nuclei.png."
    a. Read and display color image,
    b. Extract the Blue channel, save as a monochrome image, and segment and count the number of nuclei in this monochrome image.
    c. Extract the green channel and count the number of "spots" per nucleus in the corresponding blue image. Have your program display each spot with a cursor. Each spot corresponds to a protein localization.

4. **(40) Counting:** You are given an image called “clutter.png,” and asked to count the number of objects in this image. How would you do this?
5. **(40) Computing curvature:**
    a. Threshold the image called “CoffeeBean.tif”,
    b. extract the largest single connected component in this image,
    c. extract the boundary of this object and show the results, and
    d. compute the curvature along the boundary using the formula that I gave in the class,
    e. place a star at points of maximum curvature.
    f. Explain what you see.
6. **(40) Linear and non-linear embedding**: You are asked to convert a high dimensional dataset and embedded them in a lower dimensional space using linear PCA and non-linear t-SNE.
    a. Use the Swiss roll dataset: sklearn.datasets.make_swiss_roll
    b. Plot the data
    c. Apply PCA and t-SNE both and compare the results. With t-SNE, try different values of perplexity 5, 10, 20, 40, 80, and 160 and report on its impact to embedding.
    d. Run PCA and t-SNE on the wine dataset (from sklearn.datasets import load_wine): 178 samples, 13 features, and 3 classes. Show the embedding and report the results.